

# **GREEN MOBILITY & ENERGY TRANSITION: A COMPARATIVE DATA WAREHOUSING AND RDBMS ANALYSIS**

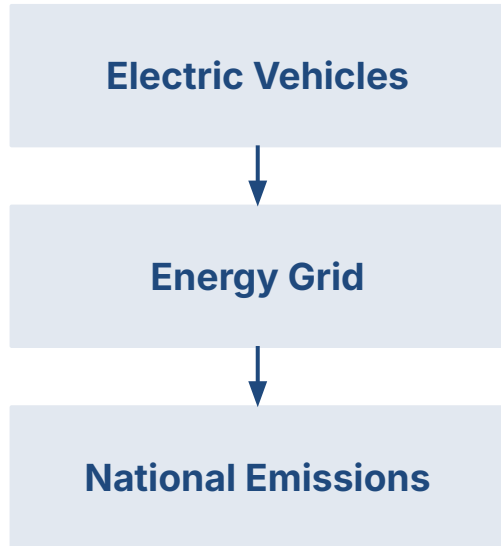
## **A Comparative Data Warehousing and RDBMS Analysis**

Students

**Casolino Ilaria**                      **2079322**  
**Giamberardini Matteo**           **2045678**

Data Management  
**a.y. 2025/2026**

# The Research Questions



- **Core Question:** Does electric vehicle adoption reduce total CO<sub>2</sub> emissions, or does it shift pollution onto fossil-fuel power generation?
- **Economic Perspective:** How does national economic capacity (GDP) affect a country's readiness for a **clean energy transition**?
- **Track 1 (DW Integration):** End-to-end ETL, Dimensional Fact Model (DFM), Star Schema implementation, and OLAP analytics.
- **Track 2 (Comparative Benchmarking):** Performance evaluation comparing a RDBMS against a Multi-Fact Star Schema.

# Data Sources

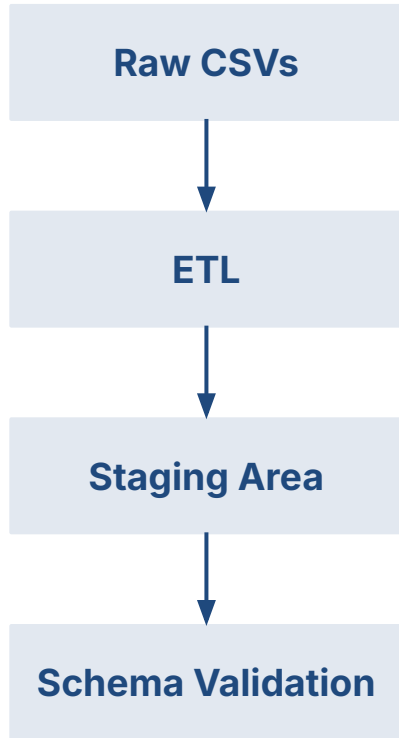
## Primary Datasets:

- **IEA EV Sales:** Contains key indicators on the electric vehicle market worldwide, including annual EV sales, stock, and charging infrastructure, broken down by vehicle type and powertrain technology.
- **OWID CO2 Emissions:** Provides data on global greenhouse gas emissions. It includes absolute emissions, source-specific outputs (coal, oil), and calculated indicators such as emissions per capita.
- **OWID Energy:** Offers comprehensive statistics on global energy consumption and electricity generation, tracking both fossil fuels and clean energy sources (solar, wind, hydro).

## Secondary Datasets (Data Cleaning & Refinement):

- **World Bank Open Data (GDP & Population):** The GDP and population values in the original OWID dataset caused inconsistencies. They were replaced with updated current USD metrics from the World Bank to improve consistency and data completeness across countries.
- **ISO Country & Continent Mapping:** The original IEA dataset lacked continent-level geographic attributes. This ISO lookup table mapped ISO codes to continents, enriching datasets with continent and continent Code to enable regional roll-up queries.
- **Ember Global Electricity Generation:** Several power generation fields in the OWID Energy dataset contained significant gaps and missing entries. The Ember dataset was integrated during ETL to fill these missing values, ensuring full coverage for electricity generation indicators.

# Data Staging, Cleaning & Transformation Strategy



## Temporal & Geographic Standardization:

- Filtered data to historical records only (2010-2023).
- Standardized country names and mapped 3-letter ISO codes across all datasets.
- Removed non-national regional aggregates (**dropna** on **iso\_code**), resulting in 50 fully mapped countries.

## Structural Pivoting:

- Converted IEA long format (**parameter** column) into wide relational format with explicit metric columns.

## Handling of Missing Values:

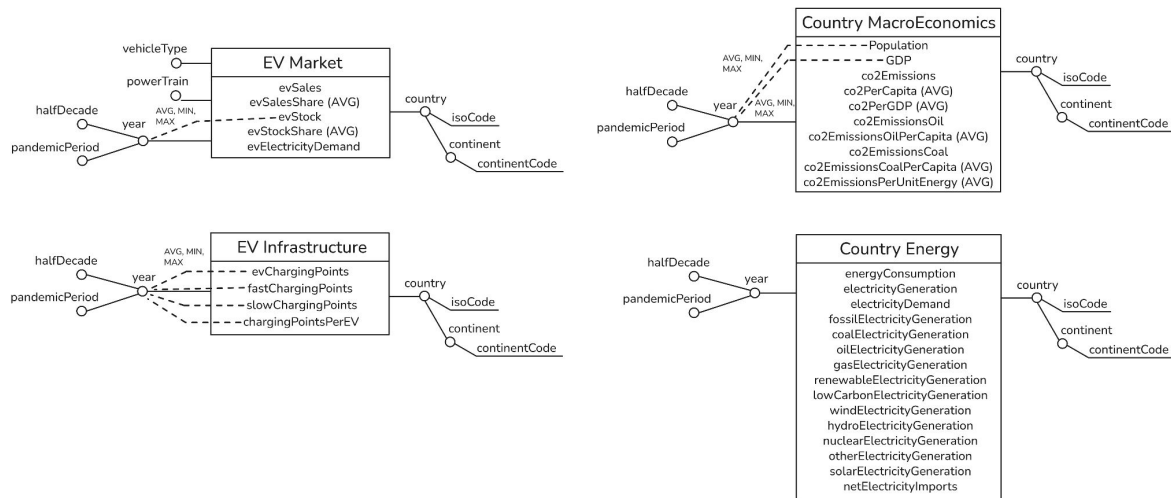
- **EV Metrics:** Imputed missing vehicle combinations to **0.0** (reflecting zero active market share).
- **Socio-Economic & Energy Metrics:** Retained as **NULL** so relational aggregation functions like **AVG()** remain statistically accurate.

# Conceptual Design: Dimensional Fact Model (DFM)

**Flow Measures: Fully additive** across all dimensions (e.g., *EV sales*, *CO<sub>2</sub> emissions*, *electricity demand/generation*).

**Level Measures: Additive across space, but not time.** They can be summed across different entities for the same period (e.g., combining Germany and France's EV stock for 2020), but not across different time periods (e.g., *EV stock*, *population*, *GDP*).

**Unit Measures: Non-additive** because they represent ratios or percentages (e.g., *EV sales share*, *CO<sub>2</sub> per capita*). They should be aggregated using functions like average, minimum, or maximum (AVG, MIN, MAX), and **never** summed (SUM).



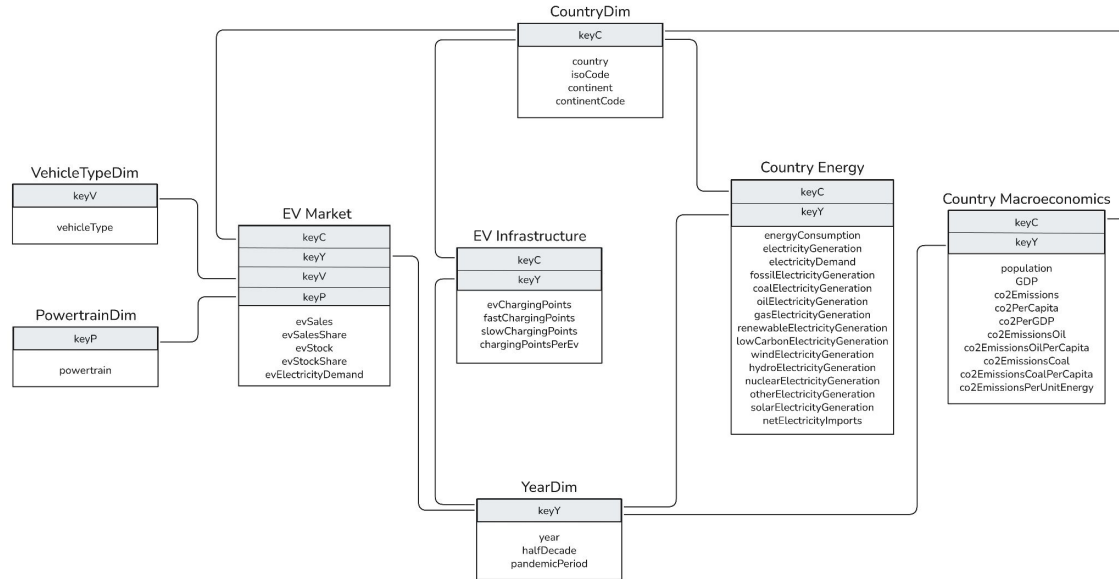
# Logical Schema Design: Star Schema

## Dimensions

- **CountryDim (keyC):** Stores country details, ISO codes, and continents.
- **YearDim (keyY):** Stores years with pre-calculated attributes.
- **VehicleTypeDim (keyV):** Vehicle categories (Cars, Buses, Vans, Trucks).
- **PowertrainDim (keyP):** Powertrain technologies (BEV, PHEV, FCEV).

## Fact Tables

- **EVMarket:** EV sales indicators.
- **EVInfrastructure:** Charging infrastructure indicators.
- **CountryEnergy:** Electricity generation by energy source.
- **CountryMacroeconomics:** emissions and macroeconomic indicators



# OLAP Queries & Analytical Insights

## EV Sales by Continent and Year (ROLLUP)

This query computes the total number of **EV sales** for each continent across different years. It helps us analyze regional adoption trends over time.

**ROLLUP(cont.continent, y.year):** This extension of GROUP BY creates a hierarchical summary. It automatically calculates three levels of aggregation in a single query:

- Detailed sales per continent and year.
- Subtotals for each continent (summed across all years).
- A grand total of global EV sales across all continents and years.

EV sales in Europe experienced massive growth after 2019, jumping from ~599k in 2019 to over 1.4 million in 2020, and reaching 3.29 million by 2023.

```
SELECT
  cont.continent,
  y.year,
  COALESCE(SUM(m.evSales), 0) AS total_ev_sales
FROM (
  SELECT DISTINCT continent
  FROM CountryDim
) cont
CROSS JOIN YearDim y
LEFT JOIN CountryDim c
  ON c.continent = cont.continent
LEFT JOIN EVMarket m
  ON m.keyC = c.keyC
  AND m.keyY = y.keyY
GROUP BY ROLLUP(cont.continent, y.year)
ORDER BY cont.continent, y.year;
```

|    | continent ▲ | year ▲ | total_ev_sales |
|----|-------------|--------|----------------|
| 53 | Europe      | 2017   | 310227         |
| 54 | Europe      | 2018   | 417951         |
| 55 | Europe      | 2019   | 599354         |
| 56 | Europe      | 2020   | 1412663        |
| 57 | Europe      | 2021   | 2362300        |
| 58 | Europe      | 2022   | 2744003        |
| 59 | Europe      | 2023   | 3290987        |
| 60 | Europe      | (NULL) | 11773151       |

# OLAP Queries & Analytical Insights

## Vehicle Type x Powertrain Cross-Analysis (CUBE)

This query analyzes EV sales by combining vehicle categories (Cars, Buses, Vans, Trucks) with powertrain technologies (BEV, PHEV, FCEV). It answers which specific vehicle segment is leading market adoption.

**CUBE(v.vehicleType, p.powertrain):** Unlike ROLLUP (which is hierarchical), CUBE generates all possible combinations of subtotals for the specified dimensions:

- Totals grouped by Vehicle Type and Powertrain.
- Subtotals grouped only by Vehicle Type (across all powertrains).
- Subtotals grouped only by Powertrain (across all vehicle types).
- The overall Grand Total.

Cars (BEV + PHEV) dominate overall sales. Battery Electric Vehicles (BEVs) are the primary driver within both Buses (~580k out of 627k) and Cars (~27.9M out of 40.3M).

```
SELECT
  v.vehicleType,
  p.powertrain,
  SUM(evSales) AS total_sales
FROM EVMarket m
JOIN VehicleTypeDim v ON m.keyV = v.keyV
JOIN PowertrainDim p ON m.keyP = p.keyP
GROUP BY CUBE(v.vehicleType, p.powertrain)
ORDER BY v.vehicleType, p.powertrain;
```

|    | vehicletype ^ | powertrain ^ | total_sales ^ |
|----|---------------|--------------|---------------|
| 1  | Buses         | BEV          | 580645        |
| 2  | Buses         | EV           | 0             |
| 3  | Buses         | FCEV         | 8503          |
| 4  | Buses         | PHEV         | 38245         |
| 5  | Buses         | (NULL)       | 627393        |
| 6  | Cars          | BEV          | 27979143      |
| 7  | Cars          | EV           | 0             |
| 8  | Cars          | FCEV         | 66511         |
| 9  | Cars          | PHEV         | 12317126      |
| 10 | Cars          | (NULL)       | 40362780      |



# OLAP Queries & Analytical Insights

## Charging Infrastructure Growth (LAG)

This query tracks the annual expansion of EV charging points per country to measure year-over-year growth and identify key inflection points in infrastructure deployment.

**LAG(evChargingPoints):** A window function that retrieves data from a previous row within the same result set.

**OVER (PARTITION BY c.country ORDER BY y.year):** Defines how the window function operates:

- **PARTITION BY c.country:** Resets the calculation for each country independently.
- **ORDER BY y.year:** Ensures rows are ordered chronologically so LAG() fetches the charging points count from the immediately preceding year.

Subtracting LAG(evChargingPoints) from the current year's count gives the exact yearly growth.

In Italy the infrastructure acceleration started picking up speed after 2015. (e.g., +360 points in 2015 vs +700 points in 2016).

```
SELECT
  c.country,
  y.year,
  evChargingPoints,
  LAG(evChargingPoints) OVER(
    PARTITION BY c.country
    ORDER BY y.year
  ) AS previous_year,
  evChargingPoints -
  LAG(evChargingPoints) OVER(
    PARTITION BY c.country
    ORDER BY y.year
  ) AS yearly_growth
FROM EVInfrastructure i
JOIN CountryDim c ON i.keyC = c.keyC
JOIN YearDim y ON i.keyY = y.keyY;
```

|     | country | year | evchargingpoints | previous_year | yearly_growth |
|-----|---------|------|------------------|---------------|---------------|
| 163 | Italy   | 2013 | 1406             | 1401          | 5             |
| 164 | Italy   | 2014 | 1410             | 1406          | 4             |
| 165 | Italy   | 2015 | 1770             | 1410          | 360           |
| 166 | Italy   | 2016 | 2470             | 1770          | 700           |
| 167 | Italy   | 2017 | 2790             | 2470          | 320           |
| 168 | Italy   | 2018 | 3470             | 2790          | 680           |

# OLAP Queries & Analytical Insights

## Core Hypothesis Test (Drill-Across Query)

This is the main analytical query of the project. It integrates data from all three fact tables (EVMarket, CountryEnergy, and CountryMacroeconomics) into a single per-country, per-year view to test whether higher EV adoption and cleaner energy grids lead to a reduction in per capita CO<sub>2</sub> emissions.

**Drill-Across (JOIN on Dimensions):** Joins multiple fact tables using their shared dimensions (keyC for country and keyY for year). This allows us to correlate EV market share (evSalesShare), clean energy generation (renewableElectricityGeneration), and carbon emissions (co2PerCapita) without duplicating values across different tables.

Between 2015 and 2023, Italy's EV sales share grew from 0.49% to 21.4%. In the same period, CO<sub>2</sub> per capita generally trended downwards (from 5.99 to 5.29 t/capita), showing a positive correlation between green transition policies and lower emissions.

```
SELECT
  c.country,
  y.year,
  SUM(em.evSalesShare) AS evSalesShare,
  AVG(ce.renewableElectricityGeneration) AS renewableElectricityGeneration,
  AVG(cm.co2PerCapita) AS co2PerCapita
FROM EVMarket em
JOIN CountryEnergy ce
  ON em.keyC = ce.keyC
  AND em.keyY = ce.keyY
JOIN CountryMacroeconomics cm
  ON em.keyC = cm.keyC
  AND em.keyY = cm.keyY
JOIN CountryDim c ON em.keyC = c.keyC
JOIN YearDim y ON em.keyY = y.keyY
GROUP BY
  c.country,
  y.year
ORDER BY
  c.country,
  y.year;
```

|     | country | year | evsalesshare | renewableelectricitygeneration | co2percapita  |
|-----|---------|------|--------------|--------------------------------|---------------|
| 283 | Italy   | 2015 | 0.490000     | 108.89000000000000             | 5.99780000... |
| 284 | Italy   | 2016 | 1.980000     | 108.01000000000000             | 5.95740000... |
| 285 | Italy   | 2017 | 1.440000     | 103.89000000000000             | 5.87750000... |
| 286 | Italy   | 2018 | 2.300000     | 114.41000000000000             | 5.82940000... |
| 287 | Italy   | 2019 | 3.110000     | 115.83000000000000             | 5.68630000... |
| 288 | Italy   | 2020 | 8.280000     | 116.90000000000000             | 5.09090000... |
| 289 | Italy   | 2021 | 16.600000    | 116.33000000000000             | 5.68090000... |
| 290 | Italy   | 2022 | 15.700000    | 100.46000000000000             | 5.76330000... |
| 291 | Italy   | 2023 | 21.400000    | 116.52000000000000             | 5.29450000... |

# Relational Approach: Normalized RDBMS Schema

## Design and Characteristics

**Normalized Design:** Stores each entity in a separate table to completely avoid duplicate data.

**Optimized for OLTP:** Built for standard database operations like inserting, updating, or deleting single rows quickly.

**Analytical Drawback:** Queries run slower because every calculation requires joining multiple tables together.

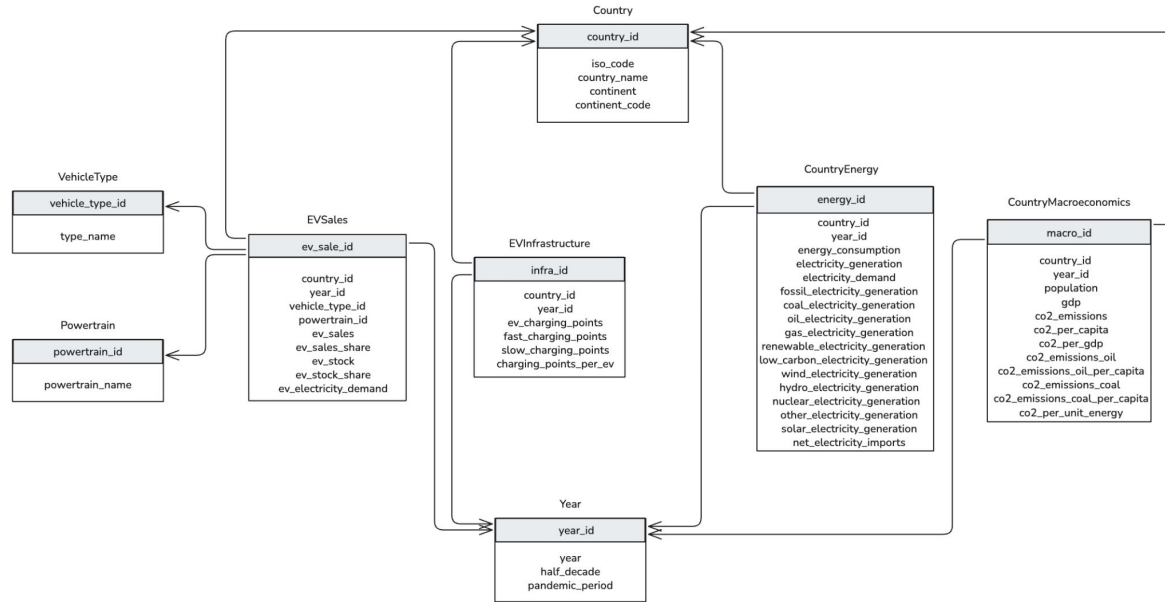
## Table Breakdown (8 Tables)

**4 Entity Tables:** Store dimensions with unique surrogate primary keys.

Country, Year, VehicleType, Powertrain (using *SERIAL* primary keys)

**4 Transactional Association Tables:** Map interactions and socio-economic metrics back to the main entities via strict foreign keys.

EVSales, EVInfrastructure, CountryEnergy, CountryMacroeconomics



# Benchmark Setup & Methodology

## 1. Environment

**DBMS Engine:** PostgreSQL 16 hosting both the Relational RDBMS and the Data Warehouse.

**Harness:** Custom Python 3 script ([benchmark.py](#)) to automate query execution.

**Execution:** Automated query execution loops with direct database drivers to minimize wrapper overhead.

## 2. Measurement Tool

**Timing Isolation:** Uses PostgreSQL EXPLAIN ANALYZE to measure planning time and execution time separately.

**Buffer Tracking:** Measures memory and buffer usage for each query run.

**Averaged Benchmark:** Runs every query 10 times.

```
EXPLAIN (  
  ANALYZE,  
  BUFFERS,  
  FORMAT TEXT  
) ;  
  
SELECT * FROM ...
```

## 3. Workload

**11 Analytical Queries:** Tested queries Q1 to Q11, covering multi-fact joins, window functions (RANK, LAG), and roll-ups (CUBE).

**Key Metrics:** Planning Time (ms), Execution Time (ms), Total Time (ms), and Speedup Factor ( $T_{RDBMS} / T_{DW}$ ).

**Statistical Protocol:** Executed 10 runs per query on both databases to smooth out OS cache variance.

# Execution Time Comparison (RDBMS vs. Data Warehouse)

## Normalized RDBMS

**30.73 ms**

Total execution time

## Data Warehouse

**22.15 ms**

Total execution time

## DW Advantage

**10 / 12 Queries**

Faster execution speed

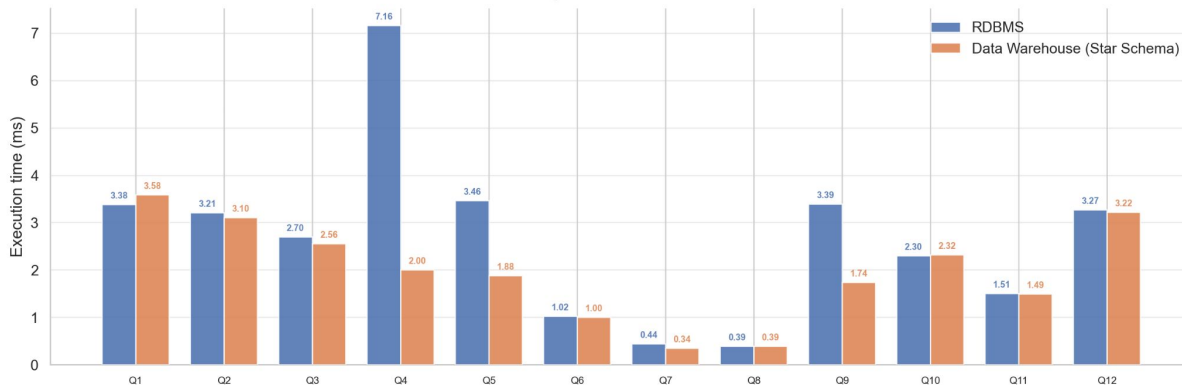
## Key Execution Highlights

**Biggest Time Savings:** Q4 (Top Countries by EV Stock) and Q5 (Vehicle Type Analysis) show the largest execution speedup in the Data Warehouse.

**Lightweight Query Parity:** For simple queries running under 3 ms (like Q1, Q2, Q6, Q8, Q10, Q12), both databases show almost identical execution times (~1.0x ratio).

**Main Efficiency Driver:** The Data Warehouse speeds up query execution by eliminating complex multi-table joins.

Chart 1 — Query Execution Time: RDBMS vs DW



# Query Planning Time Analysis

## Normalized RDBMS

**2.75 ms**

Total planning time

## Data Warehouse

**2.32 ms**

Total planning time

## Planning Speedup

**1.18x**

Star schema reduces the number of potential join permutations the PostgreSQL planner must evaluate.

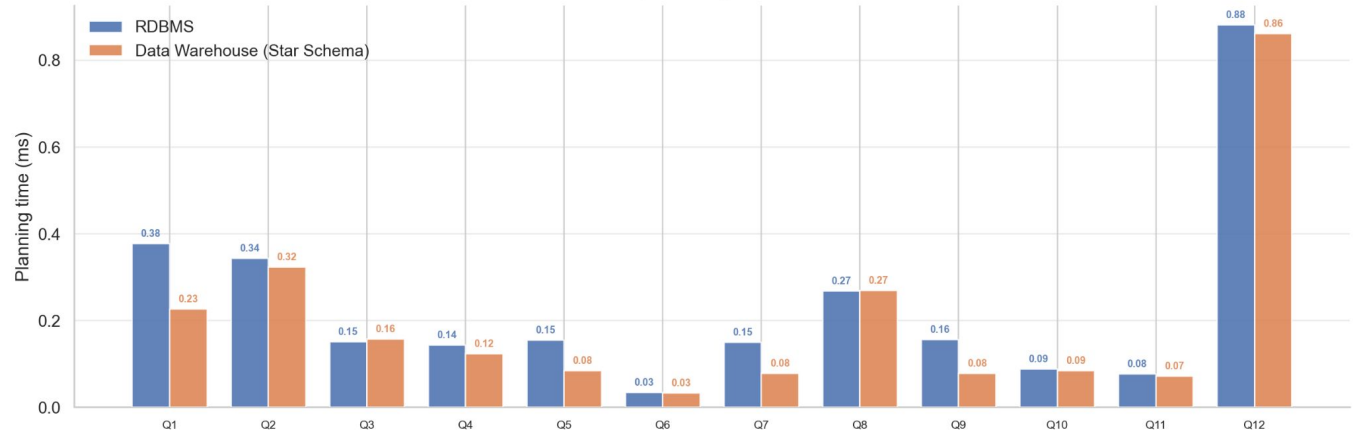
## Key Architectural Takeaways

**Simpler Join Trees:** The Star Schema reduces table join paths, helping the PostgreSQL optimizer build execution plans faster.

**Complex Multi-Fact Joins (Q12):** Planning time reaches its highest value on **Q12** (EV Adoption & CO2 Impact, 0.88 ms vs 0.86 ms) because joining 3 fact tables requires more planner calculations.

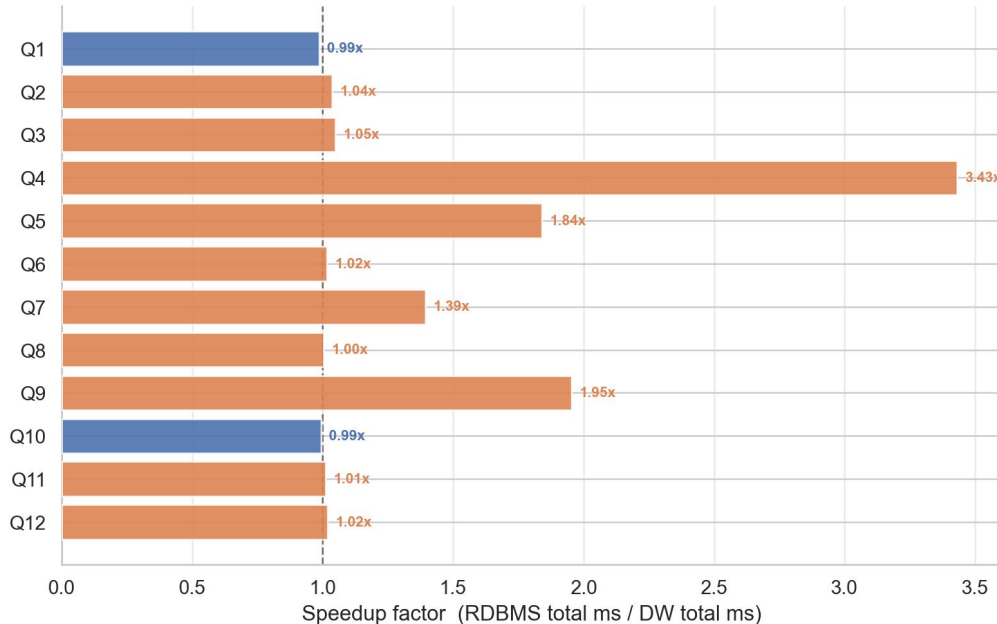
**Overall Trend:** Query planning is consistently lower or equal in the Data Warehouse across all 12 queries.

Chart 2 — Query Planning Time: RDBMS vs DW



# Data Warehouse Speedup Factor (Speedup = $T_{RDBMS} / T_{DW}$ )

Chart 3 — DW Speedup over RDBMS (total time)



## Overall Cumulative Speedup

**1.37x** overall speedup for the Data Warehouse

## Query Speedup Categories

### High Speedup (>1.5x)

**Q4** (EV Stock Ranking): **3.43x** speedup via RANK()

**Q9** (Continent x Pandemic Period): **1.95x** speedup via GROUPING SETS

**Q5** (Vehicle Type Analysis): **1.84x** speedup via CUBE operator

### Moderate Speedup (1.1x – 1.4x)

**Q7** (Infrastructure Growth): **1.39x** speedup via LAG()

### Parity (0.99x – 1.05x)

**Q1, Q2, Q3, Q6, Q8, Q10, Q11, Q12**: Almost identical performance on lightweight queries.

# Time Breakdown & Fixed Engine Overhead

## Technical Insights

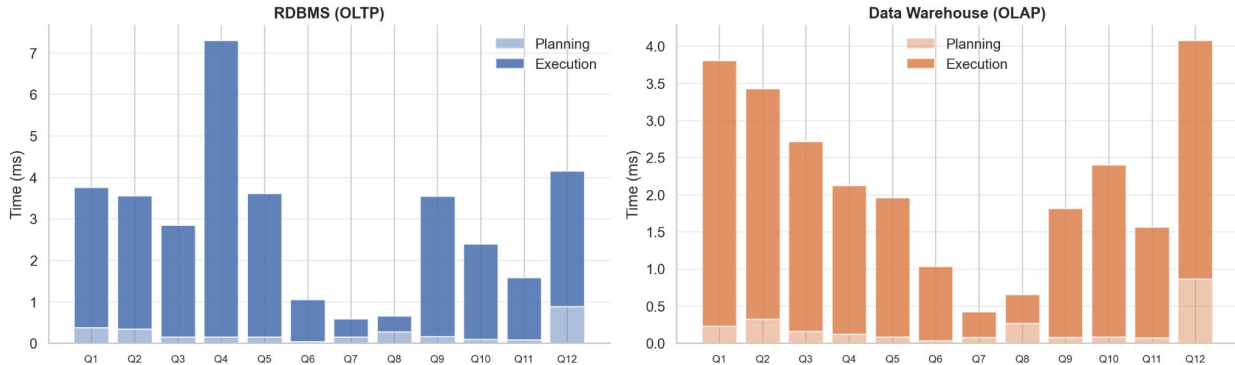
**Stacked Timing Breakdown:** Shows how Planning Time (*light shade at the bottom*) and Execution Time (*dark shade on top*) make up total query duration.

### Why Simple Queries Are Comparable (<3 ms)

- In fast queries like **Q6** (1.05 ms) and **Q7** (0.59 ms), execution finishes so quickly that fixed PostgreSQL engine overhead (parsing, buffer checks) dominates total time.
- Since our cleaned dataset fits completely in memory, basic joins run instantly on both database designs.

**Where DW Dominates:** For complex analytical rollups (like **Q4** and **Q5**), execution time dominates the query duration, allowing Star Schema optimizations to provide major time savings.

Chart 4 — DBMS vs DW Total Time





# Complete Query Performance Summary

Normalized RDBMS

33.48 ms

Total benchmark time

Data Warehouse

24.47 ms

Total benchmark time

DW Speedup

1.37x

Overall execution speedup

| ID    | Query Description                         | RDBMS Plan (ms) | RDBMS Exec (ms) | DW Plan (ms) | DW Exec (ms) | RDBMS Total (ms) | DW Total (ms) | DW Speedup |
|-------|-------------------------------------------|-----------------|-----------------|--------------|--------------|------------------|---------------|------------|
| Q1    | EV Sales (Continent x Year)               | 0.377           | 3.381           | 0.227        | 3.582        | 3.758            | 3.809         | 0.99x      |
| Q2    | Renewable Electricity vs CO <sub>2</sub>  | 0.344           | 3.208           | 0.323        | 3.104        | 3.552            | 3.427         | 1.04x      |
| Q3    | Rich vs Poor Countries                    | 0.151           | 2.698           | 0.158        | 2.559        | 2.849            | 2.717         | 1.05x      |
| Q4    | Top Countries by EV Stock                 | 0.144           | 7.162           | 0.123        | 2.005        | 7.306            | 2.129         | 3.43x      |
| Q5    | Vehicle Type Analysis                     | 0.155           | 3.460           | 0.084        | 1.879        | 3.615            | 1.964         | 1.84x      |
| Q6    | Pandemic Impact                           | 0.034           | 1.020           | 0.033        | 1.003        | 1.054            | 1.037         | 1.02x      |
| Q7    | Infrastructure Growth                     | 0.150           | 0.441           | 0.078        | 0.345        | 0.591            | 0.424         | 1.39x      |
| Q8    | Green Elec & High EV Demand               | 0.268           | 0.393           | 0.269        | 0.389        | 0.661            | 0.658         | 1.00x      |
| Q9    | Continent x Pandemic Period               | 0.156           | 3.390           | 0.078        | 1.738        | 3.546            | 1.816         | 1.95x      |
| Q10   | Renewable Share %                         | 0.088           | 2.302           | 0.085        | 2.319        | 2.391            | 2.404         | 0.99x      |
| Q11   | Is EV Adoption Reducing CO <sub>2</sub> ? | 0.882           | 3.272           | 0.862        | 3.218        | 4.155            | 4.081         | 1.02x      |
| Total | Cumulative Benchmarks                     | 2.749           | 30.727          | 2.320        | 22.146       | 33.478           | 24.466        | 1.37x      |

# SQL Code Complexity & Readability Analysis

## Normalized RDBMS

**211 lines**

Total SQL across 11 queries

## Data Warehouse

**148 lines**

Total SQL across 11 queries

## Verbosity Factor

**1.43x Ratio**

RDBMS requires more SQL

## Main Structural Causes

**Native OLAP Operators:** Using CUBE and GROUPING SETS in Q5 and Q9 reduces SQL code length from 31 lines to 9 and 13 lines ([up to 3.4x code reduction](#)).

**Window Functions:** Replacing correlated self-joins with native RANK() and LAG() saves 8 to 9 lines per query.

**Code Maintainability:** Short Star Schema queries are much easier to read, audit, and modify, reducing copy-paste errors in multi-level aggregations.

## Q5 DATA WAREHOUSE (9 Lines)

```
SELECT
  v.vehicleType,
  p.powertrain,
  SUM(evSales) AS total_sales
FROM EVMarket m
JOIN VehicleTypeDim v ON m.keyV = v.keyV
JOIN PowertrainDim p ON m.keyP = p.keyP
GROUP BY CUBE(v.vehicleType, p.powertrain)
ORDER BY v.vehicleType, p.powertrain;
```

## Q5 NORMALIZED RDBMS (31 Lines)

```
SELECT
  vt.type_name AS vehicle_type,
  pt.powertrain_name AS powertrain,
  SUM(s.ev_sales) AS total_sales
FROM EVSales s
JOIN VehicleType vt ON s.vehicle_type_id = vt.vehicle_type_id
JOIN Powertrain pt ON s.powertrain_id = pt.powertrain_id
GROUP BY vt.type_name, pt.powertrain_name

UNION ALL

SELECT
  vt.type_name,
  NULL AS powertrain,
  SUM(s.ev_sales) AS total_sales
FROM EVSales s
JOIN VehicleType vt ON s.vehicle_type_id = vt.vehicle_type_id
GROUP BY vt.type_name

UNION ALL

SELECT
  NULL AS vehicle_type,
  pt.powertrain_name,
  SUM(s.ev_sales) AS total_sales
FROM EVSales s
JOIN Powertrain pt ON s.powertrain_id = pt.powertrain_id
GROUP BY pt.powertrain_name

UNION ALL

SELECT
  NULL AS vehicle_type,
  NULL AS powertrain,
  SUM(s.ev_sales) AS total_sales
FROM EVSales s

ORDER BY vehicle_type, powertrain;
```

# Memory Hits & Disk I/O Comparison

A Comparative Analysis of Shared Buffer Cache Hits and Disk I/O Overhead Across Benchmark Queries (Q1–Q12).

## Shared Buffer Hits (Cache Efficiency)

**High Cache Reuse:** Both architectures peak dramatically at Query 12 with over 2,150 buffer hits, showing strong memory caching for large queries.

**Higher Scan Frequency:** The RDBMS exhibits slightly higher buffer hit counts in queries Q1 to Q11 (e.g., Q1, Q5, Q9), which indicates that it has to scan more data blocks in memory to process joins.

## Shared Read (Disk I/O)

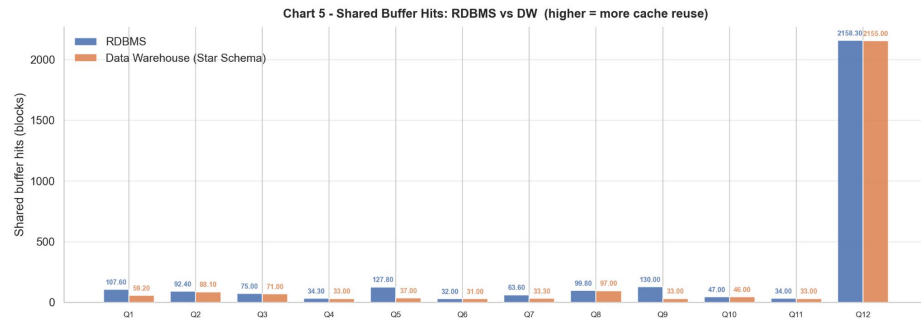
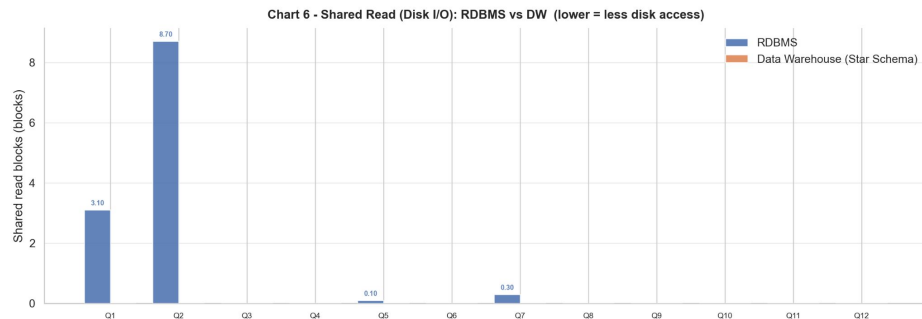
**Zero Physical Disk Access:** The Data Warehouse completely eliminates physical disk reads across all queries.

**RDBMS Latency Bottleneck:** The traditional RDBMS suffers from physical disk reads, particularly in Query 1 (3.10 blocks) and Query 2 (8.70 blocks), creating potential hardware I/O bottlenecks.

## Key Takeaways

**Disk I/O Reduction:** The Star Schema structure optimizes data storage, preventing costly physical disk lookups.

**Optimal Memory Usage:** RDBMS relies on processing more memory blocks, whereas the Data Warehouse achieves the same results with cleaner, streamlined data paths.



# Architectural Comparison & Trade-Offs

| Evaluation Dimension              | Normalized RDBMS                                                                                            | Data Warehouse (Star Schema)                                                                                    |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Core Focus                        | Optimized for transactional integrity (OLTP) and fast row-by-row operations.                                | Optimized for analytical reporting (OLAP) and fast multi-dimensional queries.                                   |
| Query Complexity                  | <b>High verbosity.</b> Requires multiple UNION ALL blocks and self-joins for rollups.                       | <b>Low verbosity.</b> Uses native CUBE, GROUPING SETS, and window functions.                                    |
| Query Performance                 | Slower on complex analytical queries ( <b>33.48 ms</b> total benchmark time).                               | <b>Up to 3.43x faster</b> (24.47 ms total time, simpler join structures).                                       |
| Data Redundancy                   | <b>Zero redundancy.</b> Fully normalized schema eliminates duplicate data.                                  | <b>Controlled redundancy.</b> Uses denormalized dimensions to prioritize query speed.                           |
| Data Pipeline (ETL)               | <b>Simpler ingestion.</b> Direct data loading using Python scripts ( <a href="#">load_db.py</a> ).          | <b>Dedicated ETL Pipeline.</b> Requires staging, key mapping, and dimension loading ( <a href="#">etl.py</a> ). |
| Memory Hits & Disk I/O Comparison | Suffers from disk I/O latency (up to <b>8.70 blocks read</b> ) and higher buffer hits during complex joins. | <b>Zero</b> physical disk reads and optimized cache efficiency.                                                 |

# Conclusions

## Superior Performance

- Star Schema achieved an overall **1.37x cumulative speedup** across all benchmarked queries.
- Executed faster in **10 out of 12 queries** compared to the normalized database.
- Reached up to **3.43x speedup** on window ranking queries (Q4).
- **Eliminates disk I/O completely** (0 reads) by leveraging efficient memory access patterns.

## Power of OLAP Syntax

- Computes multi-dimensional rollups in a single pass using **CUBE** and **GROUPING SETS**.
- Eliminates heavy self-joins with window functions (**RANK**, **LAG**).
- Reduced overall SQL code complexity (148 lines vs 211 lines).

## Answering our Research Question

- Simplifies multi-fact joins between EV market sales, grid electricity sources, and CO<sub>2</sub> emissions.
- Helps evaluate whether EV adoption effectively cuts emissions based on grid cleanliness.

## FINAL VERDICT

**The ROLAP Data Warehouse Star Schema is the optimal architecture for green mobility analytics, delivering faster performance and maintainable SQL.**